

AI (Agentic Systems)

From 01-June-2026 to 10-June-2026

LangChain vs LangGraph:

Concepts:

- What LangChain is — chains, agents, and tools as linear pipelines
- What LangGraph is — graph-based orchestration built on top of LangChain
- Why LangGraph exists — handling cycles, state, and complex control flow that chains can't
- Key differences — LangChain's sequential model vs LangGraph's node-edge graph model
- StateGraph — the core primitive in LangGraph
- Nodes — Python functions that read and write to shared state
- Edges — directed connections between nodes defining execution flow
- State schema — defining shared state with TypedDict
- When to use LangChain vs LangGraph — simple pipelines vs stateful, multi-step agents
- Compiling a graph — `graph.compile()` and invoking it with `graph.invoke()`

Workflows in LangGraph: Sequential & Parallel:

Concepts:

- Sequential workflows — chaining nodes one after another with direct edges
- `add_edge()` — connecting nodes in a fixed order
- `START` and `END` — LangGraph's built-in entry and exit points
- State passing — how each node receives and returns updated state
- Parallel workflows — running multiple nodes simultaneously from a single node
- `add_edge()` with multiple targets — fanning out to parallel branches
- Fan-in — merging outputs from parallel branches back into a single node
- Reducer functions — combining parallel outputs using `operator.add` or custom merge logic
- Annotated state fields — using `Annotated[list, operator.add]` for parallel writes
- Visualising graph structure — `graph.get_graph().draw_mermaid()`

Task:

- **Parallel Document Analyser:** Build a LangGraph workflow that takes a single input document (plain `.txt` file). In the first node, load and prepare the text. Then fan out in parallel to three nodes simultaneously — one that generates a summary, one that extracts key topics, and one that performs a sentiment analysis — each making its own LLM call. A final merge node combines all three outputs and prints a unified report to the

terminal. Log the full graph execution trace and the final merged report to a timestamped `.txt` file. All credentials in `.env`.

Workflows in LangGraph: Conditional & Iterative:

Concepts:

- Conditional edges — routing to different nodes based on state values
- `add_conditional_edges()` — defining a router function that returns a node name
- Router functions — inspecting state and returning the next node as a string
- Branching patterns — if/else logic baked into the graph structure
- Iterative workflows — graphs that loop back to a previous node
- Cycles in LangGraph — how LangGraph safely supports loops that LangChain cannot
- Loop termination — using state flags or counters to break out of cycles
- Reflection loops — a node generates output, another critiques it, the loop repeats until quality threshold is met
- `should_continue` pattern — the standard conditional edge used to exit loops
- Combining conditional and iterative logic — building self-correcting pipelines

Task:

- **Self-Correcting Essay Writer:** Build a LangGraph workflow that accepts an essay topic from the terminal. A writer node drafts an essay. A critic node scores it out of 10 and provides feedback. A conditional edge checks the score — if below 7, route back to the writer node with the critique included in state; if 7 or above, route to an END node. Cap the loop at 5 iterations using a counter in state. Print each draft and its score to the terminal, and save the final accepted essay along with all iteration logs to a timestamped `.txt` file. All credentials in `.env`.

Persistence in LangGraph:

Concepts:

- What persistence is in LangGraph — saving graph state across invocations
- Why persistence matters — enabling multi-turn conversations and resumable workflows
- `MemorySaver` — in-memory checkpointer for development and testing
- `SqliteSaver` — file-based persistence using a local SQLite database
- `PostgresSaver` — production-grade persistence with a Postgres backend
- `thread_id` — the key concept for isolating separate conversation sessions
- `config = {"configurable": {"thread_id": "..."}} — passing thread context to invoke()`
- Checkpoints — how LangGraph snapshots state at every node execution
- Resuming from a checkpoint — re-invoking a graph and continuing from saved state

- `graph.get_state()` and `graph.get_state_history()` — inspecting saved checkpoints

Task:

- **Persistent Multi-Turn Chatbot:** Build a LangGraph chatbot with `SqliteSaver` as the checkpoint, backed by a local `chat_history.db` file. On startup, prompt the user for a `thread_id` — a returning user enters their existing ID to resume their session, a new user picks a new one. Each user message is added to state and passed to an LLM node that always has access to the full conversation history. Print conversation history on session start. Add a `/history` command to display all past messages for the current thread, and a `/threads` command to list all active thread IDs stored in the database. All credentials in `.env`.

Streaming in LangGraph:

Concepts:

- What streaming is — receiving output token-by-token or node-by-node instead of waiting for full completion
- Why streaming matters — responsiveness in user-facing applications
- `graph.stream()` — streaming graph execution event by event
- Stream modes — `"values"` (full state after each node), `"updates"` (only the delta), `"debug"` (verbose internal events)
- Token-level streaming — using `astream_events()` for character-by-character LLM output
 - `on_chat_model_stream` events — filtering the event stream for LLM token chunks
 - Async LangGraph — using `async def` nodes and `asyncio` for non-blocking streaming
 - Streaming with persistence — combining `MemorySaver` and `astream_events()` together
 - Displaying streamed output — printing tokens in-place with `print(chunk, end="", flush=True)`
 - Handling stream interruptions — graceful error handling in async streaming loops

Task:

- **Live Streaming RAG Terminal:** Extend the Day 4 persistent chatbot to support full token-level streaming. Use `astream_events()` to intercept `on_chat_model_stream` events and print each token to the terminal as it arrives — character by character, with no newline until the full response completes. Also stream node-level progress updates (e.g. `[Retrieving...]`, `[Generating...]`) by printing node names as the graph transitions between them using `"updates"` mode alongside token streaming. Log the

complete final responses (not individual tokens) to a timestamped session file. All credentials in `.env`.

MCP (Model Context Protocol):

Concepts:

- What MCP is — a standard protocol for connecting LLMs to external tools and data sources
- Why MCP exists — replacing one-off tool integrations with a universal interface
- MCP architecture — Host, Client, and Server roles
- MCP Servers — lightweight processes that expose tools, resources, and prompts over a standard interface
- Transport types — `stdio` (local subprocess) and `SSE` (HTTP-based remote servers)
- Tools in MCP — callable functions exposed by a server that an LLM can invoke
- Resources in MCP — file-like data sources the LLM can read (files, DB records, API responses)
- Prompts in MCP — reusable prompt templates defined server-side
- Connecting MCP to LangGraph — using `langchain-mcp-adapters` to wrap MCP tools as LangChain tools
- `MultiServerMCPClient` — connecting to multiple MCP servers in one LangGraph agent

Task:

- **MCP-Powered Research Agent:** Build and run two local MCP servers using `mcp` and `FastMCP` — one that exposes a `search_web(query)` tool (stubbed to return mock search results from a `.json` file) and one that exposes a `read_file(path)` tool (reads from an allowed directory). Connect both servers to a LangGraph `ReAct`-style agent using `MultiServerMCPClient`. Accept a research question from the terminal. The agent must autonomously decide which tools to call, in what order, to answer the question. Print every tool call and its result, then print the final synthesised answer. Log the full tool-use trace to a `.txt` file. All credentials in `.env`.

Human in the Loop:

Concepts:

- What Human-in-the-Loop (HITL) means in LangGraph — pausing graph execution for human input
- `interrupt_before` — halting the graph before a specified node executes
- `interrupt_after` — halting the graph after a specified node executes
- `NodeInterrupt` — raising an interrupt programmatically from inside a node

- `graph.invoke()` returning mid-graph — understanding the interrupted state object
- Resuming after human input — calling `graph.invoke(None, config=...)` to continue from a checkpoint
- Editing state before resuming — using `graph.update_state()` to inject human corrections
- Approval workflows — a node proposes an action, human approves or rejects before execution
- Human feedback injection — adding a human message to state during an interrupt
- Combining HITL with persistence — why `SqliteSaver` or `MemorySaver` is required for interrupts

Task:

- **Human-Approved Action Agent:** Build a LangGraph agent that accepts a high-stakes task from the terminal (e.g. "Draft and send a project update email"). The agent has three nodes — a planner node that produces a structured action plan, a human approval node that interrupts and prints the plan asking `Approve? (yes/edit/reject)`, and an executor node that carries out the approved plan. If the human types `edit`, capture their correction and use `graph.update_state()` to inject the revised plan before resuming. If `reject`, route to END. Log the original plan, human decision, any edits made, and the final execution result to a timestamped file. All credentials in `.env`.

Short-Term and Long-Term Memory in Agents:

Concepts:

- The two types of agent memory — short-term (within a session) and long-term (across sessions)
- Short-term memory — state stored in the graph's `StateGraph` and persisted via checkpoints per `thread_id`
- Message trimming — using `trim_messages()` to prevent context window overflow in long conversations
- Summarisation as short-term memory management — condensing old messages into a running summary node
- Long-term memory — facts that persist beyond individual threads and are available to all sessions
- LangGraph `Store` — the built-in key-value interface for long-term memory (`InMemoryStore`, `AsyncPostgresStore`)
- `store.put()` and `store.search()` — writing and retrieving long-term memories by namespace
- Memory namespaces — organising memories by `user_id`, category, or topic

- Semantic search over memories — using embedding-based `store.search()` to find relevant past facts
- Combining short-term and long-term memory — using checkpointers for session state and `Store` for persistent facts

Task:

- **Dual-Memory Personal Assistant:** Build a LangGraph assistant that maintains both memory types simultaneously. Short-term memory: use `SqliteSaver` with message trimming — keep only the last 10 messages in active state, summarising older messages into a single summary node that prepends context. Long-term memory: use `InMemoryStore` with a `user_id` namespace — whenever the user states a personal fact (e.g. "I prefer Python", "My deadline is Friday"), a memory extraction node detects and stores it via `store.put()`. On every new session start, retrieve all stored long-term memories with `store.search()` and inject them into the system prompt. Add a `/memories` command to print all stored long-term facts. All credentials in `.env`.

Multi-Agent Systems:

Concepts:

- What a Multi-Agent System is — multiple specialised agents collaborating to complete complex tasks
- Why multi-agent architecture — parallelism, specialisation, and overcoming single-agent context limits
- Supervisor pattern — a central orchestrator agent that delegates subtasks to specialised worker agents
- Swarm pattern — agents that hand off control to one another peer-to-peer without a central controller
- Agent handoff — transferring control and state from one agent to another using `Command(goto=...)`
- `Command` object — LangGraph's primitive for dynamic routing and state updates from within a node
- Shared vs isolated state — deciding what state each sub-agent can read and write
- Inter-agent communication — passing results between agents through shared graph state
- Sub-graphs — compiling individual agents as their own `StateGraph` and embedding them as nodes in a parent graph
- Tracing multi-agent execution — using LangSmith or verbose logging to follow cross-agent control flow

Task:

- **Multi-Agent Research & Report System:** Build a supervisor-pattern multi-agent system with one supervisor agent and three specialised worker agents — a Web Researcher (searches and retrieves facts), a Data Analyst (processes any structured data or numbers from the research), and a Report Writer (composes the final output). Accept a broad research topic from the terminal. The supervisor breaks the topic into subtasks and routes each to the appropriate worker using `Command(goto=...)`. Workers return their output to the supervisor which assembles the final report. Print each agent's name and output as it executes. Save the final compiled report and the full delegation trace to a timestamped `.txt` file. All credentials in `.env`.

Safety Guardrails and Observability:

Concepts:

- What safety guardrails are in agentic systems — preventing harmful, off-topic, or policy-violating outputs
- Input guardrails — validating and filtering user messages before they reach the LLM node
- Output guardrails — checking LLM responses before returning them to the user
- Implementing guardrails as LangGraph nodes — a dedicated safety check node with conditional routing
- Prompt injection detection — identifying attempts to override system instructions in user input
- LangSmith — Anthropic-independent observability platform for LangChain and LangGraph
- `LANGCHAIN_TRACING_V2` and `LANGCHAIN_API_KEY` — enabling LangSmith tracing via environment variables
- Traces, runs, and spans — understanding LangSmith's data model for debugging agent execution
- Custom metadata and tags — annotating runs with `run_name`, `tags`, and `metadata` for filtering in LangSmith
- Session reporting — generating a local human-readable log of all guardrail events, questions, answers, and sources cited
-

Task:

- **Guardrailed Observability-Ready RAG Agent:** Take the RAG pipeline from the provided course syllabus (load, chunk, embed, persist, retrieve, generate) and wrap it with a complete safety and observability layer using LangGraph. Add an input guardrail node that blocks prompt injections and off-topic queries (anything unrelated to the loaded documents), routing blocked requests to a polite rejection node. Add an output

guardrail node that scans the LLM's response for hallucination signals (answer not grounded in retrieved chunks) and flags them with a warning prefix. Enable full LangSmith tracing with a descriptive `run_name` per session. Generate a local timestamped session report listing every question asked, the guardrail decision (pass/block/flag), the final answer, and all cited sources. All credentials in `.env`.

Final Project — LangGraph Multi-Agent Intelligence Terminal (Terminal Based):

The engineering team needs a terminal application that brings together every concept covered across the ten days into a single working system. It must accept raw files (PDFs, TXTs, CSVs) from a designated input folder, automatically load and chunk them using appropriate LangChain document loaders and text splitters, embed and persist all chunks into a Chroma vector store, and expose a conversational interface where the user can ask questions in a continuous terminal loop.

The application must be built as a supervisor-pattern multi-agent LangGraph system where a central Supervisor Agent dynamically delegates to four specialised worker agents — a Retriever Agent that fetches relevant chunks with similarity scores, an Analyst Agent that processes and structures the retrieved content, a Writer Agent that composes the final grounded answer, and a Safety Agent that checks every input for prompt injections and every output for hallucinations before anything is printed to the user. The graph must demonstrate all four workflow types — sequential, parallel, conditional, and iterative — naturally woven into the pipeline.

Every session must maintain short-term memory using a checkpoint so conversations are resumable by thread ID, and long-term memory using a store so user preferences and stated facts persist across sessions and are injected automatically into every new session. All LLM output must stream token by token to the terminal with visible node-level progress indicators. At least one external tool must be exposed and consumed via a local MCP server. High-stakes actions must pause for human approval before execution using LangGraph interrupts. Full LangSmith tracing must be enabled throughout. On session exit, a timestamped report must be saved logging every question asked, every safety decision made, every answer returned, and every source cited — with all credentials secured in a `.env` file and no hardcoded values anywhere in the codebase.